

BUILDING THE CENTER OF EXCELLENCE FOR POWER PLATFORM

UNDERSTANDING THE AGENTIC SOLUTION ARCHITECTURE

**A PRACTICAL ARCHITECTURE GUIDE TO MICROSOFT 365 COPILOT AND
MICROSOFT COPILOT STUDIO**

WHITE PAPER - AUGUST 2026

NICOLÁS FERNÁNDEZ

Understanding the Agentic Solution Architecture

A practical architecture guide to Microsoft 365 Copilot and Microsoft Copilot Studio

© August 2026 Nicolás Andrés Fernández. All rights reserved.

First edition, August 2026.

ABOUT THE AUTHOR

Nicolás Andrés Fernández specializes in Microsoft Power Platform, Copilot Studio, and Dynamics 365. His contributions to the Microsoft ecosystem have earned him seven consecutive Microsoft Most Valuable Professional (MVP) awards. With more than 17 years of experience, he has designed, developed, and implemented low-code and CRM solutions for organizations across multiple industries and countries. He is the author of the Building the Center of Excellence for Power Platform book series and has extensive experience creating frameworks for adoption, governance, innovation, and sustainable platform management. Nicolás regularly shares his expertise as a speaker at Microsoft and community events, while also supporting initiatives that help organizations and professionals turn technology into sustainable business value.

COLLABORATORS

Technical reviewer: Franco Martino Gagliardi & Wilmer Alcivar

This white paper is independently published by Nicolás Andrés Fernández. It is not official guidance from Microsoft or any other vendor and does not replace official product documentation, licensing terms, security guidance, professional advice, or the organization's own legal, risk, privacy, compliance, and Responsible AI decisions. Microsoft, Microsoft 365 Copilot, Microsoft Copilot Studio, Power Platform, Microsoft Agent 365, Microsoft Foundry, and other product names referenced in this document are trademarks of their respective owners. References to those products are used only for identification and educational discussion.

Minimum necessary ownership is a better architecture objective than maximum possible flexibility.

Contents

Prologue	4
Part I: The architecture	6
1. Why architecture must come before the product	7
2. A canonical architecture for agentic solutions.....	10
3. From layers to interactions: how an agentic system actually runs	15
4. The four transverse control planes	18
5. The design principle: reason probabilistically, execute deterministically.....	21
Part II: The Microsoft mappings	23
6. Mapping the canonical architecture to Microsoft Copilot Studio.....	24
7. Mapping the canonical architecture to Microsoft 365 Copilot.....	30
8. Choosing the implementation pattern: use architecture, not a feature checklist	35
Part III: Putting it to work	39
9. Reference architecture patterns	40
10. Architecture considerations that should be decided before build.....	43
11. Common architecture anti-patterns.....	47
12. A practical decision framework.....	49
Conclusion: architecture is the stable layer	51
Continue the Series.....	53
Further Reading	54

Prologue

The most important question in an enterprise agent initiative is not "Which product should we use?" It is "What architecture are we actually building?"

The word agent now covers experiences that can look similar to a user while behaving very differently underneath: a specialized Microsoft 365 assistant, a knowledge-grounded copilot, an event-driven operations agent, a tool-using reasoning system, a connected-agent topology, or a custom AI application with its own runtime. Those patterns can have different activation semantics, trust boundaries, permissions, orchestration models, operational responsibilities, and human-control requirements.

This white paper starts above the product layer. It proposes a canonical architecture that separates six functional responsibilities: Experience and Activation, Context and Grounding, Agent Runtime and Orchestration, Capabilities and Execution, Integration, and Knowledge and Enterprise Data. Four transverse control planes then constrain the complete system: Governance, Security and Compliance; Human Interaction and Oversight; Observability; and Evaluation and Responsible AI.

The model is intentionally conceptual. It does not assume that every product exposes each responsibility as a separate service or configuration object. A Microsoft capability can implement more than one layer, and a managed runtime can hide internal details that remain architecturally relevant. The purpose of the model is to make responsibilities and boundaries discussable before product-specific objects take over the design.

A second view explains component interaction. In runtimes that support iterative planning, a useful mental model is Reason, Select, Invoke, Observe, and Re-plan: it shows why tool results must return to the runtime, why knowledge and execution should use different paths, and why deterministic services remain important even when reasoning is probabilistic. This is a canonical behavior, not a claim that every implementation exposes a developer-controlled iteration loop. A Microsoft 365 Copilot declarative agent, for instance, grounds and calls external tools sequentially, with no maker-defined looped operation plan, while Copilot Studio's generative orchestration can compose several topics, tools, knowledge sources, and agents into a genuine multistep plan.

That difference is central to the paper. Microsoft 365 Copilot and Microsoft Copilot Studio are not treated as competing definitions of what an agent is. They are treated as different implementation patterns for a common set of architectural responsibilities.

Relationship to the CoE series

This paper is a companion to the Building the Center of Excellence for Power Platform series, and specifically to its two published books so far: Defining the Framework Structure for the Power Platform Center of Excellence, which establishes the broader operating model for strategy, governance, controlled innovation, adoption and training, community, and sustainable platform management, and Copilot Studio and the Future of the Power Platform Center of Excellence, which extends that framework into the agent

era through questions of autonomy, knowledge, actions, lifecycle, risk, accountability, and evolving CoE responsibilities.

The gap addressed here is narrower and more technical. Where the books explain the organizational framework around platforms and agents, this white paper focuses on the architecture inside an agentic solution and on the decisions an architect must make when that solution is implemented through Microsoft 365 Copilot, Copilot Studio, or a custom engine pattern.

What this is	What this is not
An independent architecture white paper that provides a reusable mental model, Microsoft implementation mappings, design principles, reference patterns, and an architecture review checklist.	An official Microsoft reference architecture, a licensing guide, a step-by-step configuration manual, or a replacement for current Microsoft product documentation.

Who should read it and how

The primary audience is enterprise architects, solution architects, Microsoft 365 architects, Power Platform and Copilot Studio architects, AI platform owners, Center of Excellence leaders, security and governance stakeholders, and technical decision-makers who review enterprise agent solutions.

Readers who want the core architecture can focus on Chapters 1 through 5. Chapters 6 and 7 map that architecture to Copilot Studio and Microsoft 365 Copilot. Chapter 8 compares the implementation modes through architecture responsibilities rather than features. Chapters 9 through 12 provide reusable patterns, review questions, anti-patterns, and a practical selection framework. Chapter 13 closes with the architecture principle intended to survive product changes.

ON THIS MODEL

The canonical six-layer model and four control planes are an analytical construct developed for this white paper. They are not presented as an official Microsoft architecture.

Part I: The architecture

1. Why architecture must come before the product

Enterprise AI discussions often begin with a product name. A team has access to Microsoft 365 Copilot, so it asks what agent it can build inside Microsoft 365. Another team owns Power Platform, so it starts with Copilot Studio. A development group already uses Microsoft Foundry or another AI framework, so it approaches the problem through custom orchestration. Each path can be valid, but each can also create an architectural blind spot: the platform becomes the definition of the solution.

Agentic architecture requires the opposite discipline. The solution should first be described in terms of responsibilities and interactions. Product capabilities should then be mapped to those responsibilities.

This is not merely an academic preference. It has direct consequences for security, reliability, maintainability, and cost.

Consider a seemingly simple requirement: “Create an agent that monitors incoming requests, decides which need attention, gathers supporting information, updates a business system, and asks a human for approval before a high-impact action.” At least seven architectural questions are hidden inside that sentence.

First, what activates the agent? Is a user asking for the task in a conversation, or does an external event start the process? Second, whose identity is used when data is retrieved or changed? Third, what context is available to the reasoning step? Fourth, does the agent itself contain process logic, or does it invoke deterministic workflows? Fifth, what is the authoritative data source? Sixth, how is the approval represented in the execution loop? Seventh, how will administrators reconstruct what happened after the fact?

A product-centric diagram might show a chat icon, an agent box, and a set of connectors. An architecture-centric diagram has to show activation, context assembly, reasoning, tool selection, execution, return values, observation, re-planning, identity, policy, telemetry, and human control.

That difference becomes even more important as agents become less conversational. When an agent is purely user-driven, the person naturally provides a significant portion of the control loop: they initiate the action, review the answer, ask a follow-up question, and decide whether to continue. In autonomous or semi-autonomous scenarios, those implicit controls disappear. The architecture must make them explicit.

1.1 An agent is not a model

A foundation model can interpret language, generate text, reason probabilistically, and in many cases work with multiple modalities. But a model call is not by itself an enterprise agent.

An agent adds an operational envelope around model behavior. It has a purpose, instructions, access to context, available capabilities, and a runtime that maintains state across steps. The runtime may call the model several times. It may retrieve knowledge before a model call, invoke a tool after a model decision, request approval, receive a result, and then ask the model to interpret that result.

This is why it is useful to distinguish five concepts:

- Foundation model: the probabilistic inference engine.
- Agent: the logical unit defined by objective, instructions, context, and capabilities.
- Runtime: the execution environment that holds state and coordinates calls.
- Harness: the scaffolding that mediates between the model and everything else. It constructs each prompt or context window sent to the model, parses the model's output into structured actions, and enforces retry, timeout, and loop-control behavior. It answers how a decision gets carried out mechanically against the model API, not what the agent should do next.
- Planner/orchestrator: the mechanism that selects and sequences the next action.

In a given product, these concepts may be implemented by the same managed service and may not be individually configurable. They remain architecturally distinct because they answer different questions.

If the model changes, the agent's business purpose may remain the same. If the orchestrator changes, the same tools may still be available. If the harness changes, the planner's decisions may still be correct while the mechanics of executing them differ. If a tool changes, the agent's instructions may remain stable. A good architecture makes those dependencies visible.

1.2 A tool is not an agent

The increasing use of natural-language interfaces can blur another boundary: a callable capability is not necessarily an agent.

A tool has a contract. It performs a bounded function such as retrieving a record, creating a case, calculating an amount, sending a notification, or starting a workflow. The tool can be sophisticated internally, but from the orchestrator's perspective it should be understandable through metadata: its name, purpose, inputs, outputs, permission requirements, and failure behavior.

An agent, by contrast, has discretion. It interprets an objective, reasons over context, chooses among capabilities, and can adapt its plan based on results.

This distinction becomes particularly valuable when designing enterprise-grade solutions. It allows the architecture to locate uncertainty. The uncertainty belongs mainly in interpretation and planning; the execution of high-impact business operations should be as deterministic as practical.

1.3 Knowledge is not execution

Knowledge and tools also serve different purposes.

Knowledge supplies evidence or context. A policy document, a SharePoint site, a knowledge base, or a retrieval index can help the agent determine what is true or what guidance applies. A tool changes or

retrieves operational state through a defined interface. Mixing these roles can create unsafe designs. A knowledge answer saying that a customer is eligible for a refund is not the same thing as executing the refund. A policy document explaining an approval threshold is not the approval itself.

The architecture should therefore show separate paths for:

- Grounding the decision.
- Executing the decision.

1.4 The architecture is a set of contracts

A mature agentic architecture can be understood as a set of contracts between components.

The surface sends a request or event to the runtime. Context services provide relevant state. The orchestrator receives capability descriptions. Tools expose input and output schemas. Integration components enforce connectivity and authentication. Systems of record remain authoritative. Observability captures what occurred. Evaluation compares behavior with expected quality and risk criteria. Human oversight defines where a person can approve, review, correct, or stop the process.

Thinking in contracts makes the design portable. A Copilot Studio tool, a Microsoft 365 Copilot API plugin, an MCP tool, and a custom service are not identical product objects, but they can occupy similar architectural roles.

That is the purpose of the canonical model introduced next.

2. A canonical architecture for agentic solutions

The reference model used in this paper contains six functional layers and four control planes. It is intentionally more abstract than a Microsoft product diagram. Its purpose is to make architecture decisions comparable across platforms.

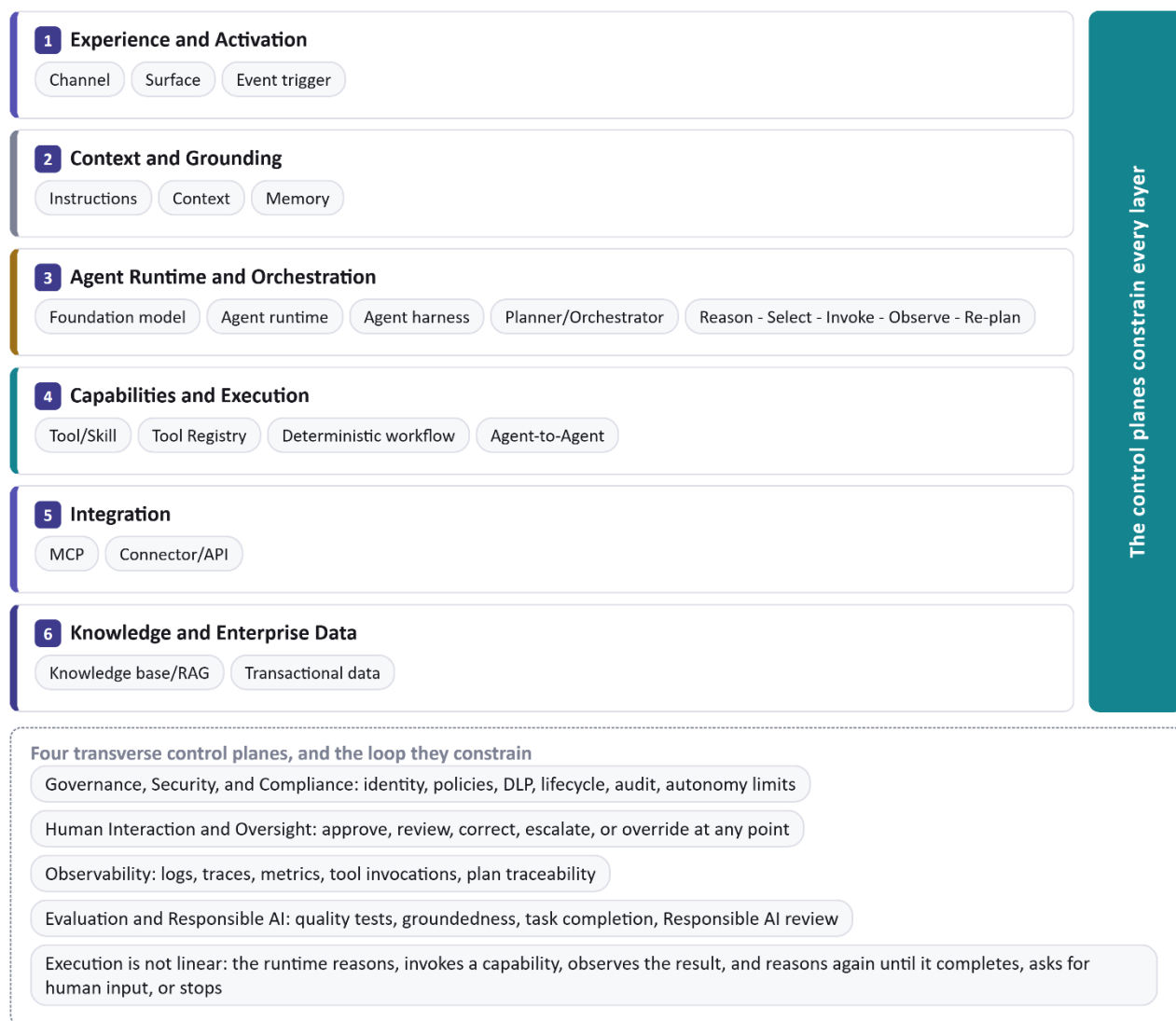


Figure 1. The six functional layers, top to bottom, with the four transverse control planes constraining the model throughout rather than acting as sequential steps.

2.1 Layer 1: Experience and Activation

The first layer answers two questions: where does the agent appear, and what starts it?

These are not the same question.

A channel is the medium through which the solution is reached: Microsoft Teams, a website, voice, email, Microsoft 365 Copilot, a custom application, or another supported interface. A surface is the concrete user experience: a chat, an embedded panel, an application page, or another interaction surface.

An event trigger is different. It activates the runtime without a conversational user message, on a new Dataverse row, an incoming event, a schedule, a file change, or another external signal. In Copilot Studio, an event trigger lets an agent using generative orchestration act autonomously in response to what just happened, rather than waiting to be asked.

Activation is a major platform-selection factor. A user-driven architecture can rely heavily on the user's conversation as the source of intent and control. An event-driven architecture must define intent and boundaries differently because there may be no human prompt at the moment execution begins.

Architects should ask:

- Is the user always present?
- Can an event start reasoning without a user?
- Does the runtime need to operate on a schedule?
- What identity is used for event-driven execution?
- What happens if the event payload is incomplete or ambiguous?
- What is the maximum safe autonomy of the event-triggered path?

This is the first reason a generic label such as “Copilot agent” is insufficient. Two agents can have the same conversational interface but fundamentally different activation semantics.

2.2 Layer 2: Context and Grounding

The second layer contains the information that conditions the agent's decisions. It includes instructions, context, memory, and retrieved grounding.

Instructions define purpose, behavior, constraints, and decision principles. They are part of the agent design, not simply prompt text. In an enterprise solution, instructions can express what the agent is allowed to do, which sources to prefer, when to ask a question, when to invoke a tool, and when to require human review.

Context is the information assembled for the current task or turn. It can include user identity, conversation history, event payloads, application state, tool results, retrieved content, and environmental metadata.

Memory is persistent state reused across interactions or tasks. It should be treated carefully because persistence changes both utility and risk. A system that remembers preferences or task state can deliver more coherent experiences, but it must also define retention, privacy, ownership, correction, and scope.

Grounding is the process of connecting reasoning to authoritative evidence. Grounding can come from explicitly configured knowledge sources, retrieval systems, Microsoft 365 context, or enterprise data made available through tools or APIs.

The architecture should make clear that grounding is not a single database call. It is a process that assembles evidence into the agent's decision context.

This layer is especially important when comparing Microsoft 365 Copilot and Copilot Studio. Microsoft 365 Copilot is inherently close to the user's Microsoft 365 work context, and Work IQ expands that concept into a broader workplace intelligence layer with context, tools, and workspaces. Copilot Studio can use explicitly configured knowledge and can also connect to Work IQ.

Work IQ, and its Copilot Studio integration in particular, is a recent surface: parts of it were in preview at the time of writing. It is treated here as an architectural role rather than as a fixed feature set, and any design that depends on it should confirm the current availability of the specific capability it needs.

The architectural question is therefore not simply “Does the product support SharePoint?” It is “What context does the runtime receive, under whose permissions, through what retrieval path, with what persistence, and with what governance?”

2.3 Layer 3: Agent Runtime and Orchestration

This is the core of the agentic system.

The runtime maintains the execution state. It receives the activation, assembles or accepts context, invokes the foundation model, calls tools, captures results, and determines when the task is complete.

The orchestrator or planner decides what should happen next.

Between the planner and the foundation model sits a fourth sub-component: the agent harness. The harness is the scaffolding that turns a planning decision into an actual model call and turns a model response into a structured action. It assembles the prompt or context window for each turn, parses the model's output into a tool call, a topic selection, or a final answer, and enforces the mechanics of the loop: retries on a malformed response, timeouts, and how many iterations are allowed before escalating. The planner decides what capability to invoke next; the harness is what actually gets that decision in front of the model and gets a usable result back out. In most managed products the harness is not exposed as a separate configuration surface, but it is still architecturally present, and it is where a surprising share of reliability problems in agentic systems originate: a well-chosen capability invoked through a brittle harness still fails.

WHERE RELIABILITY ACTUALLY FAILS

Harness reliability is easy to overlook because it rarely appears as a named setting. Malformed tool-call parsing, silent truncation of a long context window, or a retry policy that loops past the point of usefulness are harness-level failures, not planner or model failures, and they call for a different fix.

In traditional deterministic automation, the sequence is defined before runtime: step A, then B, then C, with explicit branches. In agentic orchestration, part of the sequencing can be decided at runtime based on instructions, available capabilities, context, and intermediate results.

Copilot Studio's generative orchestration illustrates this directly: it selects tools, knowledge, topics, and other agents based on their descriptions and available metadata, and can combine several capabilities to satisfy a request or respond to an event.

Architecturally, the key point is not the product name. It is that the runtime now contains a decision loop.

A useful decomposition is:

1. Reason about the current objective and state.
2. Select an appropriate capability or next conversational step.
3. Invoke the capability.
4. Observe its result.
5. Re-plan based on the new state.
6. Repeat until complete, blocked, escalated, or stopped.

This loop is the essence of agentic execution.

2.4 Layer 4: Capabilities and Execution

Capabilities are the actions available to the runtime.

They can include reusable topics or skills, tool calls, deterministic workflows, prompts, code-backed functions, other agents, approvals, business rules, and RPA or legacy automation.

A capability should be described in a way that enables selection. This is where the concept of a tool registry becomes important.

A tool registry does not have to be a literal separate database. It represents the architectural function of making capabilities discoverable through metadata. The planner needs to understand what each capability does, what inputs it requires, what outputs it returns, and under what conditions it should be used.

In Copilot Studio this relationship is explicit: generative orchestration uses names, descriptions, inputs, and outputs when deciding which topics, tools, agents, or knowledge sources to use.

The quality of tool metadata is therefore not cosmetic documentation. It is part of runtime behavior.

2.5 Layer 5: Integration

Integration answers how capabilities and external systems are exposed to the agent.

Common patterns include native connectors, custom connectors, REST APIs, workflow calls, Model Context Protocol servers, agent-to-agent protocols, and custom middleware.

MCP is particularly important because it standardizes how tools and contextual capabilities can be made available to AI systems, and both Copilot Studio and Work IQ expose governed capabilities to agents through it.

However, architects should avoid turning MCP into the architecture itself. MCP is an integration protocol. It does not replace the need to define tool ownership, identity, input validation, transactional boundaries, error handling, observability, or authorization.

The same is true of connectors and APIs. They are integration mechanisms, not substitutes for a capability design.

2.6 Layer 6: Knowledge and Enterprise Data

The lowest functional layer contains the authoritative information on which the solution depends.

It includes two broad categories.

Knowledge-oriented sources provide evidence for grounding: documents, policies, procedures, websites, indexed content, and other information used to formulate an answer or recommendation.

Transactional systems hold operational state: CRM, ERP, Dataverse, SQL databases, line-of-business systems, and external services.

The separation is essential. Enterprise systems should remain authoritative. An agent should not become an alternative system of record merely because it can remember state. Memory is not a replacement for governed transactional data.

The design should specify which source is authoritative for each entity, whether access is read-only or read/write, how freshness is handled, how retrieval is filtered by identity and permissions, how conflicting information is resolved, what data may be placed in model context, and what information must remain outside the model context and be handled through tools.

3. From layers to interactions: how an agentic system actually runs

A layered architecture is useful for classification, but it can accidentally imply a linear pipeline. Real agentic systems are more dynamic.

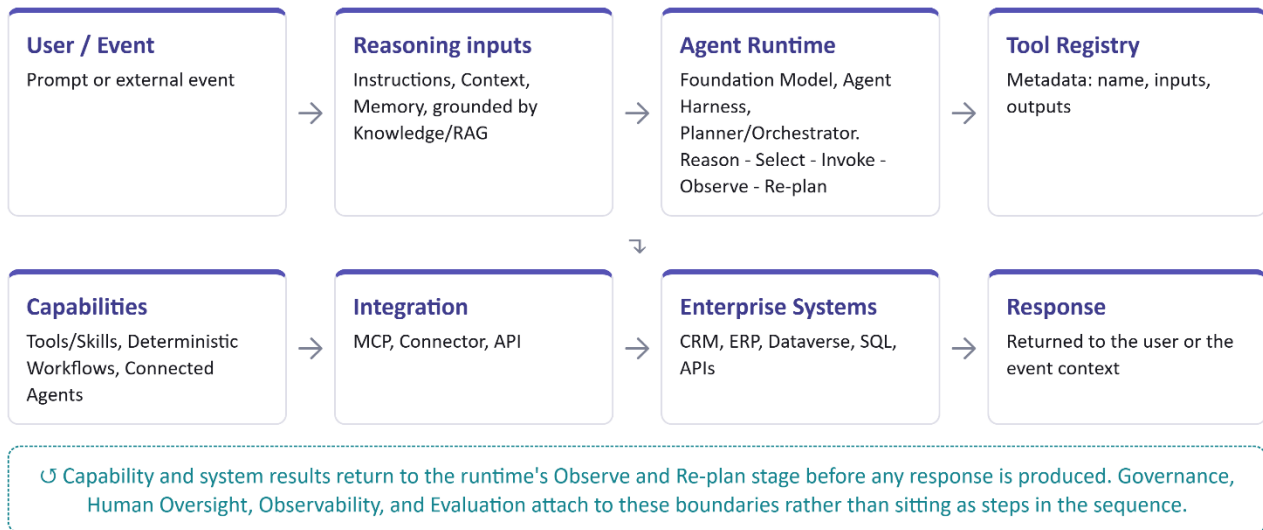


Figure 2. Component interaction: a user or event feeds the runtime, the planner selects capabilities through the tool registry, and results return for possible re-planning before the response reaches the user.

3.1 The request path

A task begins with a user request or an event.

The runtime then establishes the task state. It determines which instructions apply, what user or event context is available, whether memory is relevant, and whether knowledge retrieval is required.

The important architectural point is that context is assembled for reasoning. It is not merely displayed to the model. Good context assembly limits irrelevant information, preserves permission boundaries, and provides evidence in a form the runtime can use.

3.2 The planning path

The runtime invokes the model and orchestration logic to determine what to do.

The planner can decide that the task is already answerable from context, that additional knowledge must be retrieved, that a tool must be invoked, that another agent should handle part of the problem, or that a human must be consulted.

This is where probabilistic behavior is useful. Human requests are ambiguous. Business language is variable. The same objective can be expressed in different ways. A planner can infer intent and choose among multiple capabilities without requiring a handcrafted decision tree for every phrasing.

But this flexibility creates a new architectural dependency: the system must make the available choices understandable to the planner. That is the role of tool and agent metadata.

3.3 The invocation path

Once a capability is selected, execution should cross a clear contract.

If the capability is deterministic, it should validate inputs and execute in a predictable manner. If it changes business state, the authorization model should be explicit. If it has side effects, the design should define idempotency, retries, compensation, and error behavior as appropriate.

The agent should not be expected to compensate for weak API contracts by “reasoning harder.” This principle is particularly important for high-impact actions. A planner might probabilistically determine that a refund is appropriate, but the refund service should still enforce customer eligibility, amount limits, account state, fraud controls, and any required approvals.

3.4 The observation path

The result of a tool call is not simply an output to show the user. It is an observation that changes the state of the agentic loop.

The runtime may need to determine:

- Did the call succeed?
- Is the result complete?
- Does it contradict earlier context?
- Is another tool required?
- Should the agent ask the user for missing information?
- Has a risk threshold been crossed?
- Is human approval required?
- Can the task now be completed?

This is why the return path should be visible in architecture diagrams. Without it, the system looks like a pipeline when it is actually a stateful control loop.

3.5 The response path

Only after the runtime determines that the task is complete should it produce a user-facing response or close an event-driven execution.

In conversational scenarios, the response itself becomes new context for subsequent turns. In event-driven scenarios, completion may instead mean updating a system, emitting an event, creating a task, or waiting for an approval.

3.6 Agent-to-agent interaction

Multi-agent architecture introduces another loop.

A parent or broker agent can delegate a bounded objective to another agent. The connected agent works in its own context, uses its own capabilities, and returns a result. The parent agent then observes that result and continues its plan.

Microsoft 365 Copilot now supports connected declarative agents, with Copilot Chat acting as the broker for declared agent relationships, and a connected agent is selected based on manifest metadata such as its name and description, which reinforces how much architectural weight capability discovery actually carries.

Multi-agent design should not be adopted simply because it is available. It should be justified by a real boundary in domain, context, risk, ownership, lifecycle, or reusable capability. It is most useful when it creates a meaningful boundary: a different domain, data set, context window, team ownership, risk profile, or reusable capability.

3.7 The hidden architecture of failure

An agentic loop also needs a failure model.

A tool can time out. A connector can return an authorization error. A knowledge search can return weak evidence. Another agent can return an incomplete answer. An approval can expire. A model can select the wrong capability.

A robust design distinguishes at least four failure classes:

- Interpretation failure: the runtime misunderstands intent.
- Selection failure: the wrong capability is chosen.
- Execution failure: the selected capability fails.
- Quality failure: the process completes technically but produces an unacceptable outcome.

Each of these failure classes is answered by a different part of the control model that follows. Interpretation and selection failures are caught by evaluation and, at run time, by human oversight. Execution failures surface through observability and are contained by the execution contracts described in Chapter 5. Quality failures are the specific concern of evaluation. Governance, security, and compliance constrain what any of them can reach.

4. The four transverse control planes

The control planes are deliberately modeled outside the six functional layers. They are not sequential steps. They influence design time, deployment, and runtime.

4.1 Governance, Security, and Compliance

Governance answers what is allowed, who owns the agent, which environments and data sources are permitted, how access is granted, and how the agent is managed through its lifecycle.

Security answers who or what is acting, how that identity is authenticated, what it can access, and how threats are detected or constrained.

Compliance answers how data handling, retention, auditability, policy, and regulatory obligations are enforced.

Microsoft Agent 365 is increasingly relevant at this level. It is a control plane for observing, governing, and securing agents, with a centralized registry, lifecycle controls, access governance, and integration with Microsoft Entra, Purview, and Defender.

Agent 365 is likewise a recent control plane, and the boundary between what it governs centrally and what remains the runtime's own responsibility is still settling. The architectural distinction it makes — control plane against business runtime — is the durable part; the specific inventory of what it observes and enforces is not.

The architecture should still distinguish platform governance from runtime behavior. Agent 365 provides identity, policy, and observability guardrails, but it does not universally control the internal behavior of every runtime. That is an important architectural boundary. A control plane can constrain an agent, but a poorly designed tool, workflow, or business API can still be unsafe. Governance cannot compensate for an execution contract that fails to enforce its own business invariants.

Identity deserves particular care because an agent can act under three different models. It can act on behalf of the invoking user, inheriting that person's permissions and leaving their name on every downstream access. It can act under maker or service credentials, which decouples it from the user and concentrates privilege in a credential that outlives any session. Or it can hold its own directory identity, granted and audited as a first-class actor. The three produce different audit trails, different least-privilege designs, and different answers to what happens when the agent's owner leaves the organization.

Governance should therefore be applied at several points: agent identity model and ownership, publishing and distribution, data and connector policy, tool availability, permissions, environment and lifecycle, sharing, retention, agent-to-agent connections, autonomous triggers, and approval requirements.

4.2 Human Interaction and Oversight

Human oversight is often drawn as a final escalation box. That is too narrow.

A person can participate at multiple points: defining the original objective, providing missing context, approving a proposed high-impact action, reviewing a generated result, correcting a decision, resolving an exception, taking over a conversation, stopping or overriding an autonomous process, and validating an output after completion.

Oversight also reaches the layers below the runtime, though it is exercised differently there. At the integration layer, a person decides which connectors, MCP servers, and APIs an agent may reach at all, and approves changes to that surface. At the data layer, a person owns which sources are authoritative, which are writable, and what an agent may place in model context. These are standing decisions rather than per-execution interventions, which is why they are easy to leave out of an oversight design that only considers the moment of action.

This leads to several common oversight patterns.

Human in the loop (HITL): execution cannot proceed until a person approves or supplies a decision.

Human on the loop (HOTL): the system can act within defined boundaries while a person monitors and can intervene.

Human over the loop: policy and governance define when humans must be involved, even when they are not part of every execution.

Architects should locate these interventions explicitly. "Human oversight supported" is not enough; the design should state which action, at what risk threshold, with what context, through what interface, and what happens after approval or rejection.

When the pattern chosen is human in the loop, the value of that step depends entirely on what the person is given to work with, not just on the fact that a person is present. Approving or rejecting a proposed action means little if the reviewer cannot see why the agent chose it, what evidence it drew on, or what actually happens once they say yes. The interface around that decision point needs to surface the reasoning and the underlying data, let the person adjust or enrich a proposal rather than only accept or reject it wholesale, and make the consequence of approval visible before it is committed. A human-in-the-loop step reduced to a bare approve-or-deny button, with no visibility into how the agent got there, produces the appearance of oversight without its substance, since people end up rubber-stamping decisions they have no real way to evaluate. Getting that interaction right is a UX responsibility as much as an architectural one, and it deserves the same care as the orchestration logic it exists to supervise.

Copilot Studio can combine conversational handoff and workflow-based approvals depending on the scenario. Microsoft 365 Copilot declarative agents are naturally user-driven in many scenarios, so the user already acts as a primary operator; additional approvals can still be implemented around actions or workflows. These are different human-control models and should not be treated as equivalent.

4.3 Observability

Observability answers: What happened?

An enterprise should be able to reconstruct the relevant execution path: which agent ran, which user or event initiated it, which instructions or version applied, which tools were selected, which tool calls were made, which systems were accessed, which errors occurred, how long steps took, which other agents participated, and what outcome was produced.

Microsoft Agent 365 centralizes agent observability and, for supported Microsoft platforms, comes with built-in telemetry integration: Copilot Studio agents can automatically send agent and tool invocation telemetry into Agent 365's observability experiences, while custom or pro-code agents typically need explicit instrumentation to get the same visibility.

Observability is therefore partly a platform capability and partly an architecture discipline.

A team can have excellent logs and still have poor observability if it cannot correlate them into a coherent execution story.

4.4 Evaluation and Responsible AI

Evaluation answers a different question: Was what happened good enough?

An observable agent can still be wrong.

Evaluation should test behavior such as task completion, groundedness, correct capability selection, tool-use accuracy, answer quality, policy adherence, safety, latency, reliability, and business outcome.

Evaluation can be performed before deployment through repeatable test sets and during operation through sampled or continuous quality processes. The exact tooling varies by platform, but the architectural principle is consistent: expected behavior must be measurable.

Responsible AI is related but broader. It covers how the solution is designed, validated, governed, and monitored to manage risks associated with AI behavior and use. Responsible AI validation is required for declarative agents, and evaluation concepts are defined across Microsoft 365 agent types.

The separation between observability and evaluation is one of the most important in the model:

- Observability: what did the system do?
- Evaluation: was that behavior correct, safe, useful, and acceptable?

5. The design principle: reason probabilistically, execute deterministically

Enterprise agents create value because they can interpret ambiguity and adapt, and they create risk the moment that same adaptive behavior is allowed to leak into operations that need to be predictable. The architecture should therefore draw a clear line between decision flexibility and execution reliability, and be explicit about which side of that line each piece of behavior sits on.



Figure 3. Probabilistic reasoning and deterministic execution belong on opposite sides of the same architecture, not blended together.

5.1 Where probabilistic reasoning belongs

Probabilistic reasoning earns its keep in interpreting natural-language intent, classifying an ambiguous request, selecting relevant knowledge, choosing among capabilities, decomposing a complex objective, generating a draft, identifying missing information, deciding what question to ask next, and synthesizing the result of multiple sources. What ties these together is that they benefit from semantic understanding and rarely have one rigid correct path, so a model that can weigh context and produce a reasonable answer is doing exactly the kind of work it is good at.

5.2 Where deterministic execution belongs

Deterministic mechanisms, in turn, are the better fit for financial calculations, eligibility rules, record updates, approvals, entitlement checks, compliance rules, workflow state transitions, API validation,

irreversible operations, and regulated actions. None of this means deterministic workflows have to be simple, since a workflow can carry complex rules and integrations of its own; the distinction is that its business behavior stays controlled and testable regardless of how intricate it gets.

5.3 The orchestrator should select, not absorb, business logic

A common anti-pattern is to place too much business policy in agent instructions. Instructions are valuable for shaping agent behavior, but they are not a substitute for authoritative rules: if a policy states that transactions above a threshold require two approvals, that rule should generally be enforced in the execution mechanism rather than trusted solely to the language model's interpretation of instructions. The orchestrator can decide that an approval capability is required, but it is the approval service that should actually enforce the approval.

5.4 Determinism improves auditability

This separation also makes audits easier. If the agent proposes an action and a deterministic service validates and executes it, the organization can inspect both decisions independently, asking why the agent chose that action and whether the service executed according to policy as two separate questions with two separate answers. Without that boundary, reasoning and execution tend to become entangled in one opaque prompt-response exchange, and there is no longer a clean seam to inspect when something goes wrong.

Part II: The Microsoft mappings

6. Mapping the canonical architecture to Microsoft Copilot Studio

Copilot Studio is particularly useful for understanding the canonical model because many architectural responsibilities are exposed directly in the product.

Current architecture guidance for the platform treats channels, runtime, integrations, ALM, dialog management, language understanding, analytics, security, and AI capabilities as explicit parts of the solution architecture.

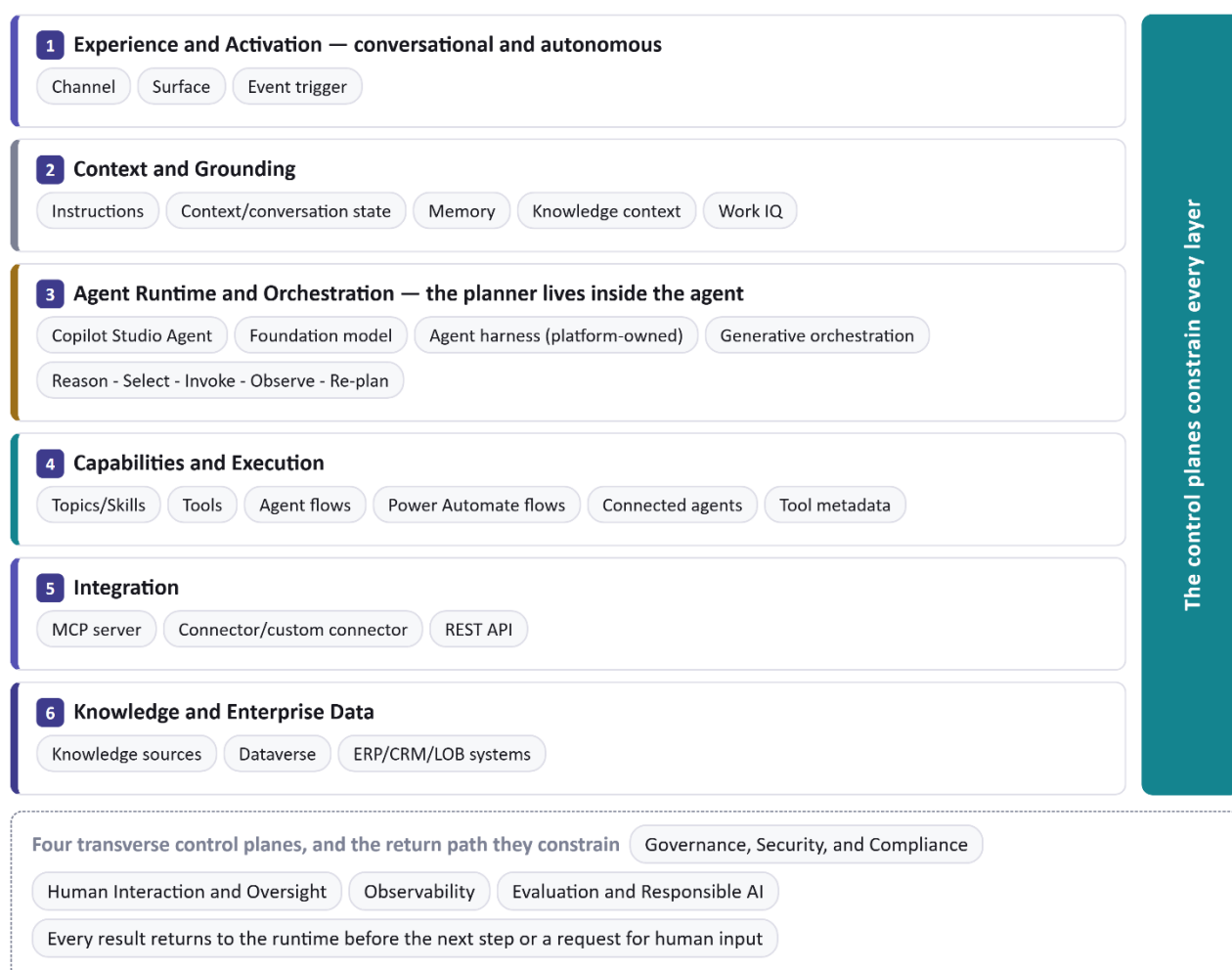


Figure 4. Each canonical layer mapped to its Copilot Studio implementation: channel and triggers, generative orchestration, topics and tools, and Dataverse as the system of record.

6.1 Experience and activation in Copilot Studio

Copilot Studio agents can be published into multiple channels and host experiences — among them Microsoft Teams, a website or custom application, Microsoft 365 Copilot, and voice, alongside further targets that vary by tenant and configuration. The precise inventory should be confirmed against current documentation for any given design, but architecturally the key point is that the agent is not inseparable

from one UI. More importantly, Copilot Studio supports both conversational activation and event-driven activation: an event trigger lets an agent with generative orchestration respond automatically when a defined event occurs, which is a significant architectural capability, since it turns the agent from a purely conversational assistant into a runtime that can participate in business process execution.

That shift brings its own responsibilities along with it. Event-driven design means thinking through trigger scoping, event payload validation, identity and credentials, duplicate-event handling, retry behavior, decision boundaries, rate and cost management, observability, and human approval thresholds, all before the agent ever fires on its own. Event-triggered agents also carry authentication considerations distinct from a conversational session, so architects should treat autonomous activation as a distinct security and operating mode, not simply "chat without a user."

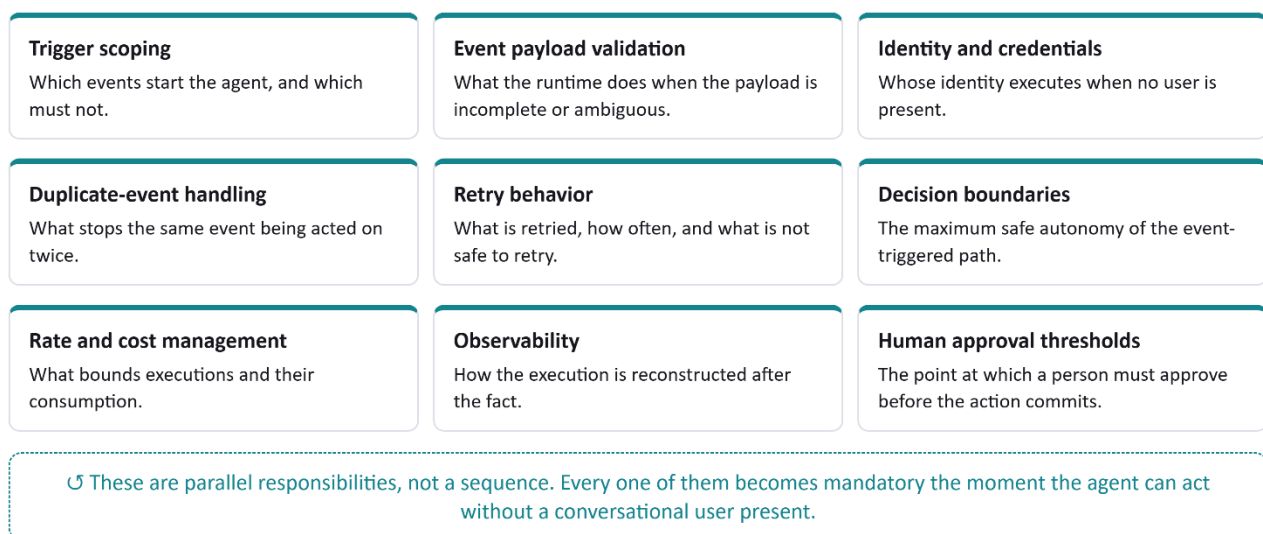


Figure 5. The responsibilities an event-driven Copilot Studio agent takes on the moment it can act without a user present.

6.2 Context and grounding in Copilot Studio

Copilot Studio can combine agent instructions, conversation state, variables and host context, knowledge sources, outputs from tools, memory capabilities where available, and Work IQ capabilities where available. Knowledge sources act as grounding material that the orchestration runtime can select from based on the user's question, but the architecture should still separate knowledge configuration from tool-based retrieval: static or semi-static reference material is usually a good fit for knowledge, while real-time account data or transactional status is better retrieved through a tool or API, because the result needs to reflect current system state and explicit authorization rather than a cached snapshot.

Work IQ adds another option on top of this: shared Microsoft 365 work context and tools that Copilot Studio can draw on directly. It is a newer surface, so it is worth isolating behind clear assumptions in the design rather than letting the whole architecture quietly depend on exactly how it behaves today.

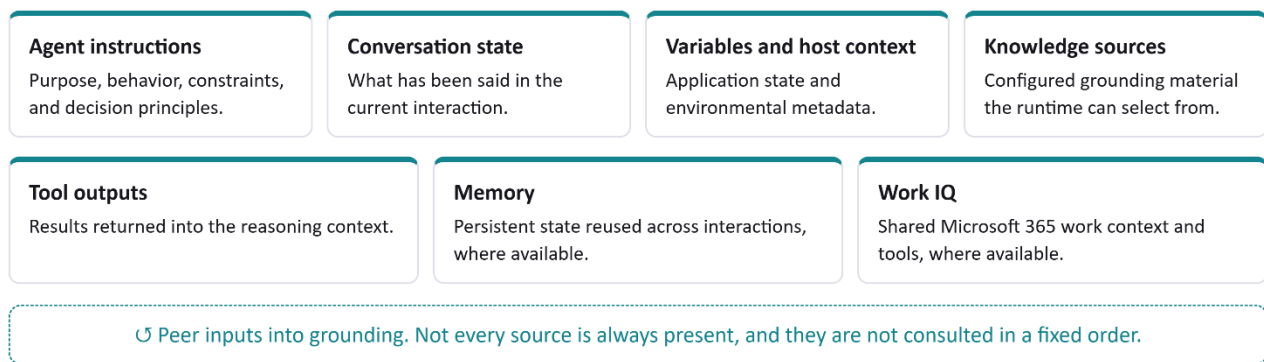


Figure 6. The context sources a Copilot Studio agent can draw on when grounding a response.

6.3 Runtime and generative orchestration

Copilot Studio's generative orchestration maps directly to the planner/orchestrator function.

It functions as an LLM-driven planning layer that interprets intent, selects appropriate tools and knowledge, and executes multistep plans, choosing among topics, tools, knowledge sources, and other agents, and combining more than one capability in a single plan when the request calls for it.

This has several architectural implications.

First, descriptions become runtime metadata. A poorly named tool is not just difficult for humans to maintain; it can be harder for the planner to select correctly.

Second, tools should have clean input and output contracts. The orchestrator can fill missing inputs conversationally, but excessive ambiguity reduces reliability.

Third, the agent's instruction set must complement, not duplicate, tool descriptions. Instructions should define behavior and constraints; tool metadata should define capability.

Fourth, testing must include orchestration decisions, not only final answers. A response can look correct while having used the wrong tool or an unsafe path.

The canonical harness sub-component maps here too, though Copilot Studio does not expose it as a separate configuration surface. The platform owns prompt construction, output parsing, and retry behavior around the foundation model; the maker configures instructions, topics, tools, and knowledge, and generative orchestration decides what to invoke, but nothing in the maker experience corresponds to editing the harness itself. That is a deliberate trade: it removes an entire failure category from the maker's responsibility, at the cost of not being able to tune it when a scenario needs harness-level control.

6.4 Topics, tools, agent flows, and Power Automate

Copilot Studio exposes several execution constructs that should not be collapsed into one category.

Topics can represent reusable conversational or behavioral blocks. Tools expose callable operations. Agent flows provide workflow-oriented execution designed to work with agents. Power Automate cloud flows can provide broader deterministic automation and integration.

From an architectural perspective, the precise product object matters less than the contract it implements. The key questions are:

- Is the capability selected by the planner?
- Is its execution deterministic?
- Does it change business state?
- What identity does it use?
- Can it fail partially?
- Can it be retried safely?
- Does it require approval?
- Is its result returned to the runtime for re-planning?

The agent should select the capability; the capability should own its business execution semantics.

6.5 Tool metadata and discovery

Tool metadata is particularly important in Copilot Studio because generative orchestration uses descriptions and schemas to decide what to call.

This leads to a practical architecture rule: treat tool metadata as part of the executable design.

Tool descriptions should state the business purpose, not just the technical implementation. Inputs should be unambiguous. Outputs should be designed so the runtime can interpret success, business result, and error state separately.

For example, a tool called `UpdateRecord` is weak. A tool called `UpdateCustomerShippingAddress` with clearly defined required inputs and a structured result is much easier to reason about, test, and govern.

6.6 Connected and child agents

Copilot Studio supports patterns in which an orchestrator can select other agents. This can be useful when:

- Domains have distinct knowledge.
- Different teams own different capabilities.
- A large agent has too many tools or sources.

- Context should be isolated.
- Different security boundaries apply.
- Specialist evaluation criteria are required.

The architecture should still define the parent-child contract. What context is passed? What is returned? Can the child act, or only advise? Which identity is used? Which agent owns the user-facing response? How are traces correlated?

6.7 Integration: connectors, REST, MCP

Copilot Studio can reach enterprise systems through connectors, custom connectors, REST calls, workflows, and MCP-based tools.

MCP is especially attractive where an organization wants a standardized tool surface across multiple agents or runtimes. It can reduce one-off integration design and support discoverability. But architects should still apply normal API architecture disciplines: authentication, authorization, schema governance, versioning, throttling, error models, and observability.

6.8 Governance and Agent 365

Copilot Studio also sits within the Power Platform governance model. Environment strategy, data policies, connectors, solution lifecycle, ALM, and administrative controls remain relevant.

Microsoft Agent 365 adds a broader cross-platform control plane, with automatic registry and observability integration for Copilot Studio agents, including telemetry for agent and connector/tool invocations.

Architecturally, this creates two complementary governance scopes:

- Platform governance: how Copilot Studio solutions are built, connected, deployed, and managed.
- Agent control plane: how agents are inventoried, observed, governed, and secured across the enterprise.

Neither replaces the other.

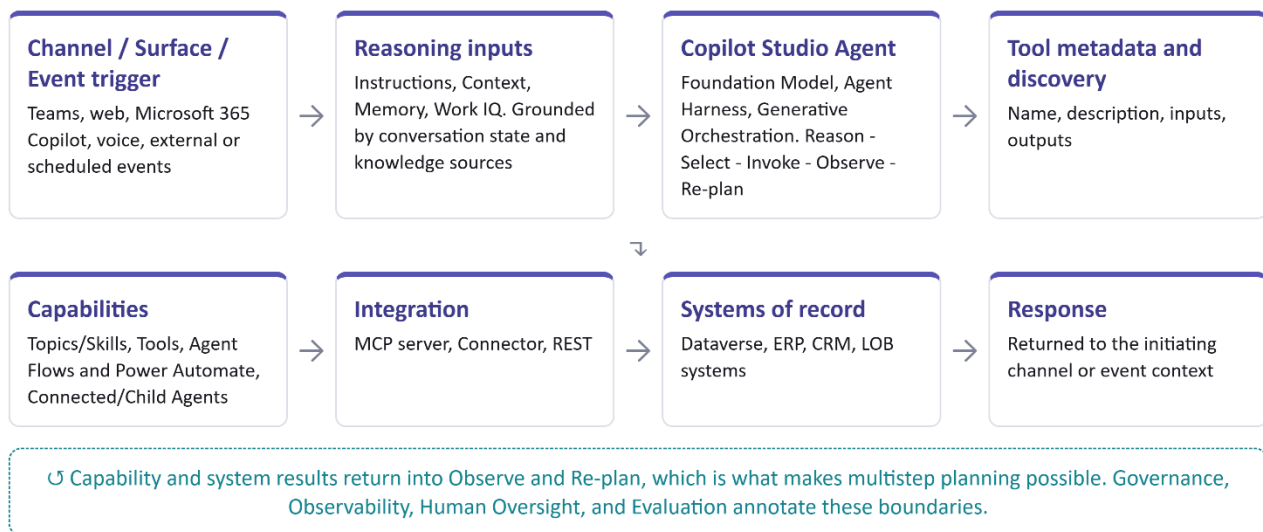


Figure 7. How a request moves through Copilot Studio's generative orchestration, selecting topics, tools, and connected agents, with results returning to support multistep planning.

7. Mapping the canonical architecture to Microsoft 365 Copilot

Microsoft 365 Copilot starts from a different architectural premise. A declarative agent specializes a Microsoft-managed Copilot runtime rather than giving the maker ownership of the core orchestrator or foundation-model layer. That distinction should drive the architecture discussion before actions, connectors, or knowledge sources are selected.

Agents in the Microsoft 365 ecosystem fall into two broad approaches: declarative agents, which use Microsoft 365 Copilot's orchestrator and models, and custom engine agents, which bring developer-controlled orchestration and models. This is an ownership boundary, not simply a development-tool choice.

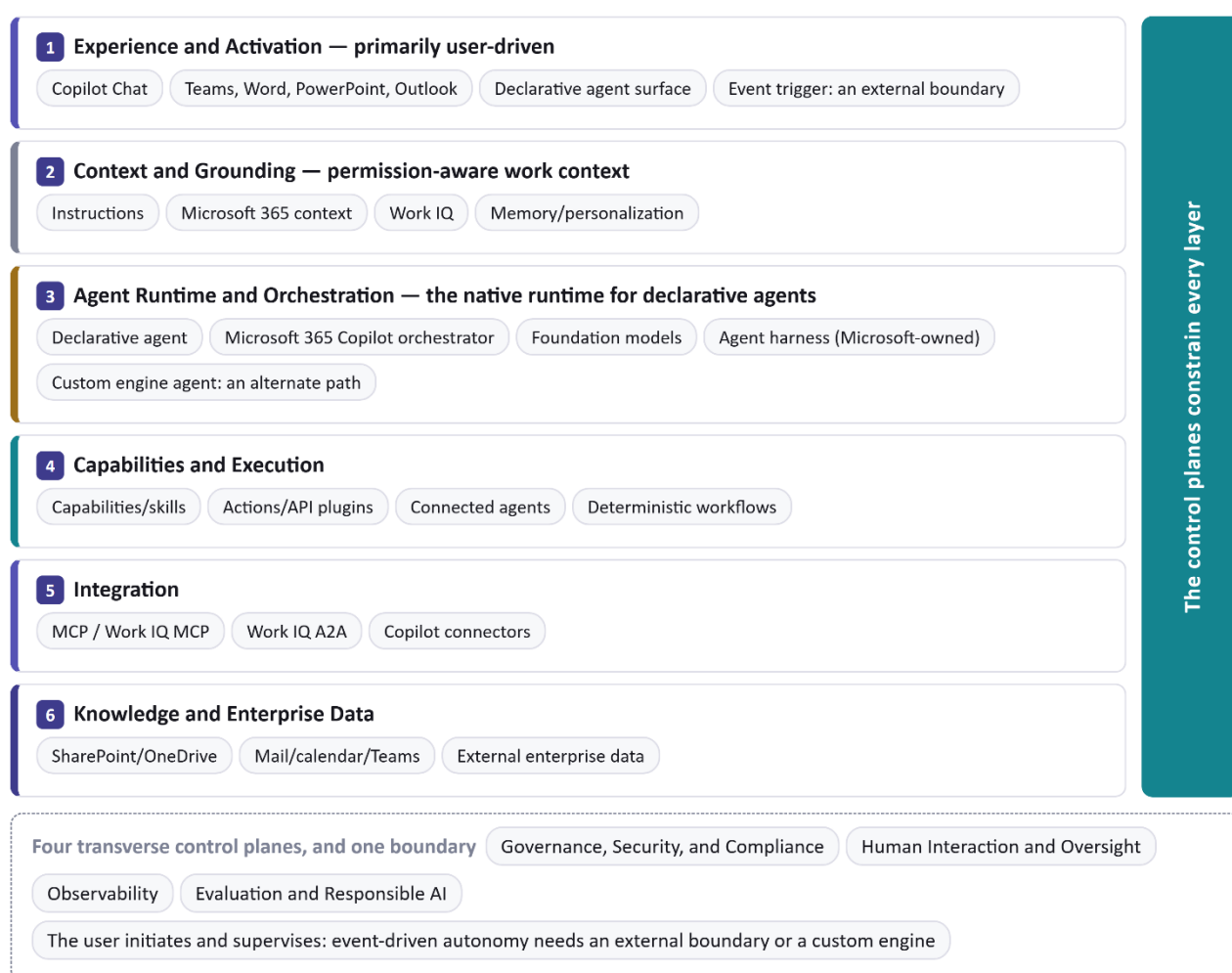


Figure 8. Each canonical layer mapped to a Microsoft 365 Copilot declarative agent, with Custom Engine Agent shown as a separate runtime-ownership path, not a component inside this runtime.

7.1 Declarative agents specialize a managed runtime

A declarative agent is an experience in which the maker provides instructions, actions, and knowledge while the agent runs on the same orchestrator, foundation models, and trusted AI services that power Microsoft

365 Copilot. The architectural value of this model is managed responsibility. The organization can create a specialized agent without owning the underlying Copilot orchestration service, model hosting, or core Microsoft 365 trust boundary.

The maker still controls consequential design decisions. Instructions define behavior. Knowledge defines evidence available for grounding. Actions expose operations. Capabilities and metadata shape what the managed orchestrator can use. Distribution, permissions, and governance determine who can use the result.

Choosing a managed runtime is not a statement about sophistication. It is a decision to accept a platform boundary in exchange for lower runtime ownership.

The harness sub-component is fully owned by Microsoft in this pattern: prompt construction against the foundation models, output parsing, and retry behavior are entirely inside the "orchestrator, foundation models, and trusted AI services" Microsoft operates on the maker's behalf. The maker never sees a harness configuration surface at all, which is the sharpest form of the managed-runtime trade-off described above.

7.2 The declarative processing model is a real architecture constraint

The canonical model in this paper uses Reason, Select, Invoke, Observe, and Re-plan to explain the general anatomy of an agentic execution loop. That model should not be projected unchanged onto every Microsoft implementation.

Declarative-agent grounding and calls to external tools are processed sequentially: developers cannot chain operations against grounding data or define looped operation plans in the declarative architecture, and orchestration control stays limited by design for complex multistep scenarios.

WORTH VERIFYING

A Microsoft 365 Copilot declarative agent should not be drawn as if the maker owns a general-purpose iterative planner. The canonical loop remains useful for understanding agentic systems, but the declarative implementation exposes a more constrained, Microsoft-managed processing model.

When the solution requires developer-controlled iterative reasoning, complex decision trees, repeated tool-result re-planning, or proactive execution, evaluate Copilot Studio generative orchestration or a custom engine pattern instead.

This distinction is one of the most important architecture decisions in the paper. It prevents a conceptual model from being mistaken for a literal product runtime.

7.3 Experience and activation are primarily user driven

Declarative agents are designed to extend Microsoft 365 Copilot and supported Microsoft 365 experiences. They are a natural fit when the user initiates work inside Copilot Chat or another Microsoft 365 surface and remains the primary operator.

Proactive interactions are not supported for declarative agents, while custom engine agents can initiate workflows and actions without direct user input, which makes activation a clear architecture discriminator.

A requirement for autonomous reaction to external business events should therefore trigger an explicit platform review. Copilot Studio provides event triggers for agents using generative orchestration, while a custom engine can implement a fully custom proactive model. The correct choice depends on the rest of the architecture, but the requirement should not be hidden behind a generic label such as "Microsoft 365 agent."

7.4 Microsoft 365 context and Work IQ

Microsoft 365 Copilot's strongest architectural advantage is proximity to permission-aware work context. Work IQ makes that capability more reusable across agent runtimes.

Work IQ is a workplace intelligence layer that combines chat, context, tools, and workspaces, exposing its capabilities through A2A, MCP, and REST. It can assemble agent-ready context across Microsoft 365 information, expose governed tools, provide persistent workspaces for long-running tasks, and centralize authorization, policy, and logging.

This means Work IQ should not be reduced to a final box after Microsoft Graph or to a synonym for a knowledge index. In the canonical model, it spans Context and Grounding, Capabilities and Execution, Integration, and parts of the control plane.

The Copilot Studio integration with Work IQ is still a newer surface, so it is worth keeping the stable design principles distinct from the parts of the implementation most likely to keep shifting.

7.5 Actions and connected agents extend the managed experience

Declarative agents can expose actions and API plugins that perform bounded operations. Architecturally, these actions occupy the same role as tools in the canonical model: they should have a clear purpose, bounded permissions, explicit inputs and outputs, deterministic business behavior where required, and appropriate user confirmation.

Microsoft 365 Copilot also supports connected declarative agents. The connection is declared in the manifest, Copilot Chat acts as the broker at runtime, connected-agent selection depends on metadata such as name and description, and communication between connected declarative agents is text-only.

Connected agents are therefore useful for domain separation and reuse, but they should not be interpreted as a way to bypass the declarative runtime's orchestration constraints. They create delegation between managed agents; they do not give the maker ownership of an arbitrary iterative execution loop.

7.6 Custom engine agents mark the runtime-ownership boundary

When the managed declarative runtime is insufficient, the architecture can move to a custom engine pattern.

A custom engine agent gives developers control over orchestration, language models, and API integrations while remaining compatible with Microsoft 365 experiences. That control supports iterative tool calling, custom model routing, specialized state management, proprietary planning logic, and other requirements that fall outside the declarative architecture.

The price is responsibility. The solution team must address hosting, instrumentation, error handling, security implementation, runtime state, model and orchestrator lifecycle, scale, availability, and incident response.

Custom engine is an architecture category, not a mutually exclusive product label. Copilot Studio is among the tools that can be used to build custom engine agents. In this paper, "Copilot Studio" normally refers to the platform-managed generative-orchestration pattern described in Chapter 6; "custom engine" refers to the case in which the solution team owns the core orchestration or model behavior, regardless of which Microsoft tooling participates in the implementation.

7.7 Governance, observability, and evaluation remain design responsibilities

Declarative agents inherit significant Microsoft 365 security, compliance, and governance capabilities. Microsoft Agent 365 adds a centralized control plane for observing, governing, and securing agents across the organization.

That managed foundation reduces operational burden, but it does not remove architecture review. The solution architect still has to decide which knowledge is exposed, which actions exist, what requires confirmation, how connected agents expand the trust boundary, how data classifications affect use, who owns the agent, how the agent is retired, and how behavior is evaluated.

Agent 365 should also be treated as a control plane rather than as the business runtime. Governance can constrain identities, access, lifecycle, policy, and telemetry, but business APIs and workflows must still enforce their own invariants.

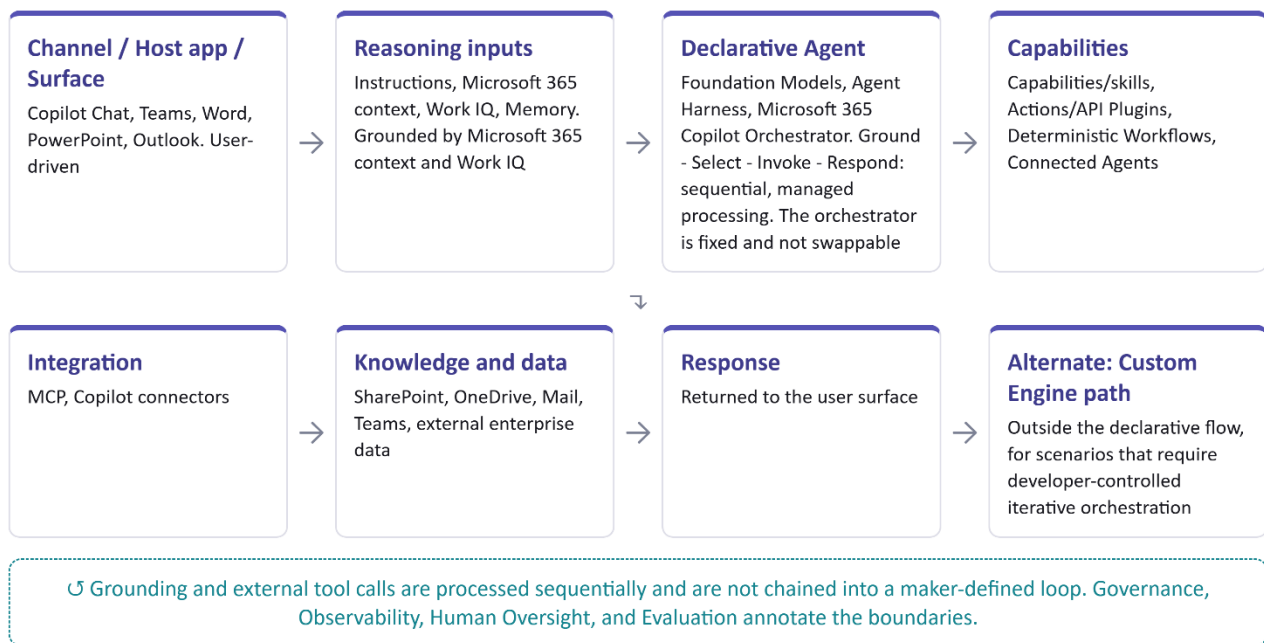


Figure 9. A declarative agent's sequential grounding and action processing, contrasted with the alternate Custom Engine path for scenarios needing iterative orchestration.

The Microsoft 365 mapping therefore preserves the canonical responsibilities while making one boundary explicit: the product owns more of the runtime than the solution team does.

8. Choosing the implementation pattern: use architecture, not a feature checklist

A feature checklist is easy to produce and quick to age. Architecture decisions are more durable because they describe where responsibility sits, how work is activated, how reasoning is controlled, and what the organization must operate.

The comparison in this chapter is between architecture modes, not three mutually exclusive commercial products. The Copilot Studio column refers to the platform-managed generative-orchestration pattern described in Chapter 6. The custom engine column refers to an architecture in which the solution team owns core orchestration or model behavior; Copilot Studio can participate in custom engine implementations.

8.1 Runtime ownership

A Microsoft 365 Copilot declarative agent uses the Microsoft-managed Copilot orchestrator and models.

A Copilot Studio generative agent uses a platform-managed runtime but gives the maker explicit control over the configured instructions, topics, tools, knowledge, event triggers, connected agents, and workflows that generative orchestration can select.

A custom engine pattern moves ownership of the orchestrator and model layer toward the solution team.

The decision is not low code versus pro code. The decision is how much runtime responsibility the scenario requires the organization to own.

8.2 Activation model

Declarative agents are strongest when the user initiates the interaction in Microsoft 365.

Copilot Studio can support the same conversational model and can also react to external events through event triggers when generative orchestration is enabled.

Custom engine patterns can implement proactive or externally orchestrated activation when the solution requires complete control.

Activation changes identity, credentials, supervision, retry behavior, and cost. It should be decided before the tool inventory is designed.

8.3 Orchestration shape

This is the most consequential distinction.

Declarative agents use a Microsoft-managed, sequential processing model for grounding and external tool calls, with limited control over complex iterative orchestration.

Copilot Studio generative orchestration can build multistep plans and select combinations of topics, tools, knowledge, and other agents based on context and metadata.

A custom engine pattern is appropriate when the solution needs its own orchestration algorithm, repeated processing loops, specialized model routing, or execution semantics that the managed platforms do not expose.

8.4 Microsoft 365 context

If the solution primarily helps a user work with Microsoft 365 information inside Microsoft 365 Copilot, a declarative agent has a natural alignment.

Work IQ weakens the old assumption that Microsoft 365 context can only be consumed through the native declarative runtime, since it is exposed through A2A, MCP, and REST, with a Copilot Studio integration as well.

The architecture question is therefore not "Do we need Microsoft 365 data?" It is "How much of the Microsoft 365 experience, trust boundary, context assembly, and managed Copilot runtime do we want to inherit?"

8.5 Deterministic process execution

A declarative agent can invoke bounded actions and API plugins, but its managed sequential processing model is a poor fit for scenarios that require developer-controlled iterative workflow orchestration.

Copilot Studio is a strong fit when generative decisions need to be combined with explicit tools, agent flows, Power Automate, connectors, approvals, and event-driven automation.

Custom engine patterns become relevant when deterministic execution requires specialized infrastructure, proprietary orchestration, or integration behavior that cannot be represented cleanly in the managed platform.

8.6 Multi-agent design

Both Microsoft 365 Copilot and Copilot Studio support forms of agent delegation, but the orchestration semantics differ.

Connected declarative agents are brokered by Copilot Chat and communicate through a constrained managed mechanism. Copilot Studio generative orchestration can select connected or child agents as capabilities in a broader multistep plan.

Choose a multi-agent topology only when it creates a real boundary in domain, context, ownership, risk, lifecycle, or reuse. An additional agent should solve an architecture problem, not merely demonstrate that delegation is technically possible.

8.7 Human oversight

Microsoft 365 declarative agents usually begin with a human already operating the experience. Confirmation and review can therefore remain close to the user.

Copilot Studio can also be conversational, but event-driven activation makes the human-control design more explicit. An autonomous trigger should define when a person must approve, review, correct, stop, or take over.

Custom engine patterns must implement their own oversight mechanisms where the host platform does not provide them.

8.8 Operational responsibility

The three modes can be summarized as follows:

Dimension	Microsoft 365 Copilot declarative agent	Copilot Studio generative-agent pattern	Custom engine pattern
Primary experience	Microsoft 365 Copilot and Microsoft 365 apps	Multiple supported channels and process-oriented scenarios	Microsoft 365, custom channels, or mixed
Runtime ownership	Microsoft-managed	Platform-managed with explicit maker configuration	Solution-team controlled
Orchestration shape	Sequential managed processing; limited iterative control	Generative multistep planning over configured capabilities	Fully custom iterative or specialized orchestration
Proactive/event-driven activation	User initiated in the declarative model	Event triggers supported with generative orchestration	Fully custom
Microsoft 365 context	Native Microsoft 365 Copilot context	Configured knowledge and tools; Work IQ integration available	Work IQ, Copilot APIs, Graph, or custom retrieval as designed

Dimension	Microsoft 365 Copilot declarative agent	Copilot Studio generative-agent pattern	Custom engine pattern
Deterministic execution	Bounded actions and API plugins	Agent flows, Power Automate, tools, connectors, and APIs	Custom services, workflows, or code
Multi-agent	Connected declarative agents through Copilot Chat broker	Connected or child agents in generative plans	Fully custom
Governance foundation	Microsoft 365 plus Agent 365	Power Platform plus Agent 365	Must be deliberately integrated with relevant control planes
Development and operations burden	Lowest runtime ownership	Moderate	Highest
Best reason to choose	Specialize Microsoft 365 Copilot without owning its runtime	Combine managed generative orchestration with events, tools, workflows, and broader integration	Own the core runtime because platform constraints are unacceptable

	Microsoft 365 Copilot declarative agent	Copilot Studio generative-agent pattern	Custom engine pattern
Experience	Native inside Microsoft 365 apps	Multiple channels, process-oriented	Microsoft 365, custom, or mixed
Activation	User initiated	User or event triggers	Fully custom
Orchestration shape	Sequential, managed processing	Generative multistep planning	Fully custom or specialized
Deterministic execution	Bounded actions, API plugins	Agent flows, Power Automate, tools	Custom services or code
Runtime ownership	Microsoft-managed	Platform-managed, maker configured	Solution-team controlled

Custom engine is an architecture pattern, not a mutually exclusive product

It can use Copilot Studio as one implementation tool. No column is the recommended one: the right choice follows from the requirements.

Figure 10. Microsoft 365 Copilot declarative agents, the Copilot Studio generative-agent pattern, and a custom-engine pattern compared across five architecture decision axes.

The objective is minimum necessary ownership. A managed runtime is strategically valuable when it satisfies the scenario; additional control is useful only when the requirements justify the additional responsibility.

Part III: Putting it to work

9. Reference architecture patterns

Architecture becomes useful when it can be reused. The following patterns show how the same canonical model can lead to different Microsoft implementations without turning product choice into the architecture itself.

9.1 Pattern A: Microsoft 365 knowledge and bounded-action agent

Scenario: A legal operations team wants an agent in Microsoft 365 Copilot that can answer policy questions from approved Microsoft 365 content and invoke a small number of well-defined case-management actions.

The user initiates the interaction. Microsoft 365 context and configured knowledge ground the response. The declarative agent runs on the managed Microsoft 365 Copilot orchestrator. API actions expose bounded operations against the case-management system. High-impact actions require explicit user confirmation. Evaluation focuses on grounded answers, correct action selection, and policy adherence.

Why this pattern fits: the experience is user driven, Microsoft 365 context is central, the action surface is small, and the scenario does not require developer-controlled iterative planning across grounding and tool results. That fits the documented strengths and processing constraints of declarative agents.

Tradeoff: the boundaries that make this pattern cheap are the same ones that cap it. Grounding and tool calls are processed sequentially, activation stays with the user, and the orchestrator is not the team's to change. If the scenario later needs the agent to react when a case is filed, or to re-plan on what a tool returned, the pattern does not extend — it is replaced.

9.2 Pattern B: Event-driven operations agent in Copilot Studio

Scenario: An operations team wants an agent to react when a high-priority case is created, gather context, classify urgency, invoke deterministic workflows, and request approval when an action exceeds a threshold.

The event activates Copilot Studio generative orchestration. The runtime uses event data, instructions, knowledge, and capability metadata to select the appropriate topics, tools, workflows, or connected agents. Deterministic agent flows, Power Automate flows, or APIs perform the consequential business operations. The result of each selected capability contributes to the runtime's plan and final response or completion path.

Why this pattern fits: event-driven activation and multistep composition are core requirements, while deterministic components remain available for business rules and state-changing actions.

Tradeoff: the moment the agent can act without a user present, the responsibilities Chapter 6 lists stop being optional — trigger scoping, event payload validation, identity and credentials, duplicate-event handling, retry behavior, decision boundaries, rate and cost management, observability, and human

approval thresholds. A conversational agent inherits most of that from the person in front of it; an event-driven one has to design every part of it.

9.3 Pattern C: Role agent with connected domain agents

Scenario: A field manager needs one front-door agent, but HR, inventory, and sales knowledge are owned by different teams.

A role agent provides the user-facing experience. Domain agents preserve focused context, ownership, and evaluation. Only task-relevant information should be passed between agents.

In a Microsoft 365 declarative pattern, Copilot Chat brokers connections between installed declarative agents and the exchange is text-based. This works well for bounded delegation and domain reuse.

In Copilot Studio, generative orchestration can select connected or child agents as part of a broader plan. This is more appropriate when delegation must participate in a multistep orchestration pattern.

Why this pattern fits: the boundary is real before the architecture creates it — HR, inventory, and sales knowledge already have different owners, different content, and different evaluation criteria. Delegation here follows an existing division of ownership rather than inventing one, which is the test Chapter 8 sets for a multi-agent topology.

Tradeoff: use another agent only when it creates a meaningful domain, context, ownership, risk, lifecycle, or reuse boundary. A bounded deterministic function is usually better represented as a tool.

9.4 Pattern D: Microsoft 365 experience with a custom engine runtime

Scenario: A specialized enterprise AI solution should be discoverable in the Microsoft 365 agent ecosystem but requires proprietary orchestration, repeated tool-result loops, specialized model routing, or infrastructure that the declarative runtime does not expose.

The Microsoft 365 experience remains valuable, but the core runtime is custom. The solution team controls orchestration, models, runtime state, tool calling, and operational behavior. Microsoft 365 context can be accessed through approved Work IQ, Copilot API, connector, or other Microsoft interfaces as appropriate.

Custom engine agents can be built with low-code Copilot Studio as well as pro-code tools and other Microsoft platforms. The architectural distinction is therefore ownership of the core runtime, not the authoring tool alone.

Why this pattern fits: the scenario's requirements — proprietary orchestration, repeated tool-result loops, specialized model routing — sit outside what any managed runtime exposes, while the Microsoft 365 experience remains where the users are.

Tradeoff: the organization gains control and assumes substantially more responsibility for security implementation, telemetry, availability, lifecycle, and incident response.

9.5 Pattern E: Agent over deterministic enterprise services

Scenario: A customer service solution must interpret complex requests, but all consequential operations are already exposed as governed APIs and workflows.

Keep those deterministic services. Add the agent as a semantic decision and orchestration layer rather than rebuilding the business process inside instructions.

THE PRINCIPLE

The most reliable enterprise pattern is often not to make every component agentic. Let the agent handle ambiguity, context, and capability selection; let deterministic services enforce business invariants and execute consequential operations.

This pattern can be implemented with either Microsoft 365 Copilot actions, Copilot Studio tools and workflows, or a custom engine, depending on activation and orchestration requirements.

Why this pattern fits: the consequential operations are already governed, so the agent adds interpretation and selection to a business process that is otherwise sound, rather than replacing it.

Tradeoff: the agent's value is capped by the quality of the existing service contracts, and weak APIs cannot be compensated for at the reasoning layer.

10. Architecture considerations that should be decided before build

The following questions are intended as an architecture review checklist. They are more durable than a product feature comparison. These questions assume a candidate pattern is already on the table. Chapter 12 provides the prior step — the sequence that selects the pattern in the first place — so a team starting from nothing should read that chapter first and return here once a candidate exists.

10.1 Purpose and outcome

- What outcome is the agent accountable for?
- Is the agent advising, deciding, acting, or all three?
- What is the boundary between the agent and the existing business process?
- What must remain deterministic?

10.2 Activation

- Is activation conversational, event-driven, scheduled, or mixed?
- Is a human present at the start?
- Can the agent act when no user is present?
- What stops repeated or duplicate execution?
- What limits autonomy?

10.3 Identity and permissions

- Is the agent acting on behalf of a user?
- Is it using maker/service credentials?
- Does it have its own agent identity?
- Which permissions are required by each tool?
- How is least privilege enforced?
- Are tool permissions different in conversational and autonomous modes?

10.4 Context and grounding

- Which instructions apply?

- Which data can enter model context?
- Which data should remain behind tool calls?
- Which knowledge sources are authoritative?
- How is freshness handled?
- Does memory exist, and what is its lifecycle?
- Is Microsoft 365 work context required?
- Is Work IQ relevant, and if so, is the required capability production-ready?

10.5 Orchestration

- Does the platform own the orchestrator or does the solution?
- Can the planner select multiple capabilities?
- How are capabilities described?
- What happens when the wrong tool is selected?
- Does the design require explicit planning constraints?
- How are long-running tasks represented?

10.6 Tool and workflow design

- Is each tool bounded to a clear business capability?
- Are input schemas explicit?
- Are outputs structured?
- Can execution be safely retried?
- Is the operation idempotent?
- What validation remains server-side?
- What approval is required?
- How are business errors differentiated from technical errors?

10.7 Integration

- Connector, custom connector, API, workflow, or MCP?
- What is the authoritative interface?

- How is authentication handled?
- How are versions managed?
- What are the latency and throttling expectations?
- Does the agent need dynamic tool discovery?
- Is A2A required, or is a tool call simpler?

10.8 Multi-agent topology

- Why is another agent better than another tool?
- Does the child agent have an independent domain or owner?
- What context is passed?
- What is the contract?
- What happens if it fails?
- How are security and observability correlated?
- Who owns the final answer?

10.9 Human oversight

- Which actions require confirmation?
- Can the user modify the proposed action?
- Can a human interrupt autonomous execution?
- What happens after rejection?
- What is the escalation path?
- Is human handoff conversational, workflow-based, or operational?

10.10 Observability

- Can every consequential action be traced?
- Can a trace correlate agent, user/event, tool, system, and result?
- Are prompts or sensitive context logged, and should they be?
- What metrics indicate health?
- Who monitors behavior?

- How long is telemetry retained?

10.11 Evaluation

- What does “good” mean for the scenario?
- What test set represents production behavior?
- How is tool selection evaluated?
- How is groundedness measured?
- How are safety and policy adherence tested?
- What causes deployment to be blocked?
- What production signals trigger re-evaluation?

10.12 Lifecycle and governance

- Who owns the agent?
- Which environment hosts it?
- How is it deployed?
- How are versions promoted?
- What happens when the owner leaves?
- How are unused agents retired?
- How are connectors and agent relationships reviewed?
- How does Agent 365 fit into centralized governance?

10.13 Entitlement, capacity, and cost

- Which licenses does each candidate pattern require, for makers and for end users?
- Is agent consumption metered, and if so, what meters it — messages, actions, sessions, or inference?
- Do any tools depend on premium connectors, and who holds that entitlement?
- What is the expected volume, and what does the pattern cost at that volume rather than in a pilot?
- What throttles or capacity limits apply, and what happens when one is reached?
- For an event-driven path, what bounds the number of executions a single event storm can trigger?
- Which of these figures must be revalidated against current licensing documentation before commitment?

11. Common architecture anti-patterns

11.1 The “LLM in the middle of everything” anti-pattern

In this design, every business rule is placed in instructions and every system call is treated as an unconstrained model decision.

The result is difficult to test and audit.

Correction: keep reasoning in the agent, but push policy enforcement and state-changing operations into deterministic services.

11.2 The “knowledge equals truth” anti-pattern

A document says what should happen, so the agent treats the document as executable policy.

Correction: use knowledge to inform reasoning; use authoritative rules and services to enforce business behavior.

11.3 The “one giant agent” anti-pattern

A single agent is given every document, every tool, and every domain.

As capability count increases, selection becomes harder, context becomes noisier, ownership becomes unclear, and evaluation becomes less focused.

Correction: reduce the tool surface, improve metadata, and introduce agent boundaries only where they create genuine domain separation.

11.4 The “multi-agent by fashion” anti-pattern

The opposite problem is splitting simple functionality into many agents.

Correction: use another agent when the delegated component needs independent reasoning, context, ownership, or lifecycle. Use a tool when the capability is a bounded function.

11.5 The “governance at deployment only” anti-pattern

The agent passes a security review before release, but no one can see what tools it calls or how its behavior changes over time.

Correction: govern the lifecycle and instrument runtime observability.

11.6 The “observability equals evaluation” anti-pattern

The team has logs and therefore assumes the agent is working well.

Correction: build testable quality criteria. Logs tell you what happened; evaluation tells you whether it was acceptable.

11.7 The “Microsoft 365 data means Microsoft 365 declarative agent” anti-pattern

Microsoft 365 context is needed, so the team automatically chooses a declarative agent without examining activation or orchestration requirements.

Correction: assess the complete architecture. Work IQ and Microsoft APIs increasingly allow Microsoft 365 context to participate in other runtimes. Choose the agent model based on runtime responsibility and interaction pattern, not data source alone.

11.8 The “Copilot Studio means chatbot” anti-pattern

Copilot Studio is treated only as a conversational bot platform.

Correction: evaluate its generative orchestration, event triggers, tools, workflows, connected agents, integration options, and Agent 365 governance as components of a broader agentic runtime.

12. A practical decision framework

The platform decision can be made more reliably by answering seven questions in sequence. Chapter 8 described the differences; this chapter turns them into a selection sequence. It runs before the design checklist in Chapter 10, which assumes a pattern has already been chosen.

Question 1: Where must the experience live?

If the primary experience is Microsoft 365 Copilot and Microsoft 365 applications, start by evaluating a declarative agent.

If the experience spans external channels, custom surfaces, process-centric execution, or several publishing targets, evaluate Copilot Studio or a custom engine pattern.

Do not let the desired chat surface determine the runtime before the rest of the architecture is understood.

Question 2: Who or what activates the work?

If a user always initiates the interaction, a Microsoft 365 declarative pattern may be sufficient.

If external business events or schedules must start the agent without a user, Copilot Studio event triggers deserve explicit consideration.

If activation is proprietary, highly distributed, or tightly coupled to custom infrastructure, evaluate a custom engine.

Question 3: Is the processing model sequential or iterative?

If the scenario can be satisfied through Microsoft-managed grounding followed by bounded actions in a sequential interaction, a declarative agent may be the simplest fit.

If the runtime must dynamically compose several topics, tools, knowledge sources, or agents into a multistep plan, Copilot Studio generative orchestration is the more natural managed pattern.

If the team must own repeated reasoning loops, custom model routing, proprietary planning, or complex state transitions, use a custom engine pattern.

Question 4: What must execute deterministically?

Identify every operation that changes material business state or enforces a rule. Put validation, approvals, calculations, authorization, and irreversible state transitions in deterministic services wherever practical.

Then ask how the chosen runtime will invoke those services. In Microsoft 365 Copilot this is typically through bounded actions or plugins. In Copilot Studio it can include tools, agent flows, Power Automate, connectors, and APIs. In a custom engine it is an explicit service contract.

Question 5: How much Microsoft 365 context is required?

If the scenario is fundamentally about a user's Microsoft 365 work and the managed Copilot experience is part of the value proposition, declarative agents are naturally aligned.

If Microsoft 365 context is one source among several, evaluate Work IQ, Copilot APIs, and Copilot Studio's integration with them rather than assuming the entire runtime must be declarative.

Question 6: How much runtime responsibility is the organization willing to own?

More control means more obligations: hosting, security implementation, monitoring, scale, availability, telemetry, testing, model management, incident response, and lifecycle.

Choose the least runtime ownership that still satisfies the activation, orchestration, integration, control, and user-experience requirements.

Question 7: Where must a person stand in the loop?

Identify every action whose consequence a person must see before it is committed, and every autonomous path a person must be able to stop. If oversight can stay with a user who is already operating the experience, a declarative pattern is sufficient. If an event-driven path must pause for approval, Copilot Studio's workflow-based approvals and conversational handoff apply. If oversight must be enforced where no host platform provides it, that obligation belongs to the solution team and should be priced into the custom engine decision.

Delegation is the second question this chapter deliberately leaves until last. Add another agent only after the six preceding answers are settled, and only for the boundaries named in Chapter 8: domain, context, ownership, risk, lifecycle, or reuse.

Conclusion: architecture is the stable layer

This agent ecosystem is evolving quickly, and product names, APIs, and management experiences will continue to change. A feature matrix built for one release will age faster than the responsibilities an enterprise agent must fulfill.

The durable layer is the architecture.

That claim applies unevenly across this document, and deliberately so. Chapters 1 through 5 and the review questions in Chapter 10 describe responsibilities that will outlast any particular release. Chapters 6 through 9 and the selection sequence in Chapter 12 map those responsibilities onto products as they stood at the verification date given in Further Reading, and should be re-checked against current documentation before they are relied on. A reader returning to this paper after a product cycle should expect the first half to hold and the second half to need confirmation.

The six functional layers in this paper separate experience and activation, context and grounding, runtime and orchestration, capabilities and execution, integration, and knowledge and enterprise data. The four transverse control planes make governance, human oversight, observability, and evaluation part of the system rather than afterthoughts.

The component-interaction view then adds the behavior that a layered diagram cannot show: requests arrive, context is assembled, reasoning occurs, capabilities are selected, deterministic services execute, results return, and the runtime decides whether the work is complete. In architectures that expose an iterative planner, that behavior can repeat. In architectures that deliberately constrain orchestration, such as Microsoft 365 Copilot declarative agents, the implementation must respect the managed processing model rather than pretending the conceptual loop is fully developer controlled.

This makes the relationship between Microsoft 365 Copilot and Copilot Studio clearer.

A Microsoft 365 Copilot declarative agent is a strong choice when the organization wants to specialize a Microsoft-managed Copilot experience with instructions, knowledge, actions, and connected agents while keeping runtime ownership low. Its sequential processing and limited orchestration control are not defects to be hidden; they are architecture boundaries that help define the scenarios for which the pattern is appropriate.

A Copilot Studio generative agent is a strong choice when the solution needs a managed platform with explicit event activation, multistep generative orchestration, selectable topics, tools, knowledge, connected agents, workflows, and broader integration.

A custom engine pattern is the next boundary when the solution team must own the orchestrator, models, repeated processing loops, or specialized runtime behavior. Copilot Studio can participate in that pattern, so custom engine should be understood as a responsibility model rather than as a competing product label.

Work IQ and Microsoft Agent 365 reinforce the architecture-first view. Work IQ makes governed Microsoft 365 context, tools, and collaboration mechanisms available across frameworks and runtimes. Agent 365 provides a broader control plane for observing, governing, and securing agents. Neither removes the need to define trustworthy tools, deterministic execution boundaries, human control points, and evaluation criteria.

WHERE TO START

Start with the canonical architecture, not the product. Define activation, context, runtime ownership, orchestration shape, capability contracts, deterministic execution, integration, authoritative data, human control, observability, and evaluation. Then map those responsibilities to Microsoft 365 Copilot, Copilot Studio, or a custom engine pattern.

When product choice follows that sequence, it becomes a consequence of architecture rather than a substitute for it.

Continue the Series

This white paper answered a focused architecture question: how to understand the responsibilities and control boundaries of an agentic solution before deciding how Microsoft 365 Copilot, Microsoft Copilot Studio, or a custom engine should implement them. The Building the Center of Excellence for Power Platform series addresses the wider framework required to govern, enable, operate, and sustain those solutions across the organization.



Defining the Framework Structure for the Power Platform Center of Excellence

[View on Amazon ->](#)

<https://www.amazon.com/dp/B0GDDRCD2C>



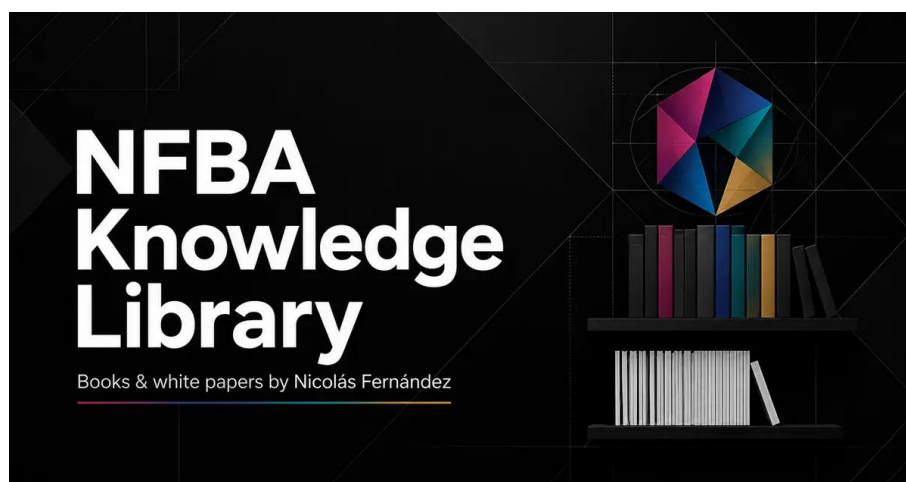
Copilot Studio and the Future of the Power Platform Center of Excellence

[View on Amazon ->](#)

<https://www.amazon.com/dp/B0H2SXQ3QQ>

Access to these and more material here:

<https://nfernandezba.github.io/Building-the-Center-of-Excellence-for-Power-Platform/>



Further Reading

Product capabilities referenced throughout this white paper were checked against Microsoft Learn on August 23, 2026. Product capabilities, schemas, licensing, regional availability, and terminology can change, so architecture and implementation decisions should always be revalidated against current official documentation and the organization's own security, privacy, legal, compliance, and Responsible AI requirements.

- Microsoft Learn. Orchestrate agent behavior with generative AI - Microsoft Copilot Studio.
<https://learn.microsoft.com/en-us/microsoft-copilot-studio/advanced-generative-actions>
- Microsoft Learn. Apply generative orchestration capabilities - Microsoft Copilot Studio.
<https://learn.microsoft.com/en-us/microsoft-copilot-studio/guidance/generative-orchestration>
- Microsoft Learn. Define your solution architecture - Microsoft Copilot Studio.
<https://learn.microsoft.com/en-us/microsoft-copilot-studio/guidance/architecture-overview>
- Microsoft Learn. Declarative agents for Microsoft 365 Copilot.
<https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/overview-declarative-agent>
- Microsoft Learn. Agents for Microsoft 365 Copilot.
<https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/agents-overview>
- Microsoft Learn. Work IQ overview.
<https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/work-iq/>
- Microsoft Learn. Work IQ in Microsoft Copilot Studio (preview).
<https://learn.microsoft.com/en-us/microsoft-copilot-studio/add-work-iq>
- Microsoft Learn. Overview of Microsoft Agent 365.
<https://learn.microsoft.com/en-us/microsoft-agent-365/overview>
- Microsoft Learn. Event triggers overview - Microsoft Copilot Studio.
<https://learn.microsoft.com/en-us/microsoft-copilot-studio/authoring-triggers-about>
- Microsoft Learn. FAQ for generative orchestration - Microsoft Copilot Studio.
<https://learn.microsoft.com/en-us/microsoft-copilot-studio/faqs-generative-orchestration>
- Microsoft Learn. Use Model Context Protocol with Microsoft Copilot Studio.
<https://learn.microsoft.com/en-us/microsoft-copilot-studio/agent-extend-action-mcp>
- Microsoft Learn. Work IQ MCP overview.
<https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/work-iq/mcp/overview>
- Microsoft Learn. Connect to other agents from a declarative agent.
<https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/declarative-agent-connected-agent>
- Microsoft Learn. Agent management overview in the Microsoft 365 admin center.
<https://learn.microsoft.com/en-us/microsoft-365/admin/manage/agent-365-overview>
- Microsoft Learn. Understand how content is shared with agents in Microsoft 365.
<https://learn.microsoft.com/en-us/microsoft-agent-365/share>
- Microsoft Learn. Build agents with Copilot Studio and Agent 365.
<https://learn.microsoft.com/en-us/microsoft-agent-365/builder/overview>
- Microsoft Learn. Knowledge sources summary - Microsoft Copilot Studio.
<https://learn.microsoft.com/en-us/microsoft-copilot-studio/knowledge-copilot-studio>
- Microsoft Learn. Custom engine agent architecture.
<https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/custom-engine-agent-architecture>
- Microsoft Learn. Custom engine agents for Microsoft 365.
<https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/overview-custom-engine-agent>
- Microsoft Learn. Declarative agent architecture.
<https://learn.microsoft.com/en-us/microsoft-365-copilot/extensibility/declarative-agent-architecture>